Criptografía y Seguridad

Criptografía:
MACs y Cifrado Autenticado

Criptosistema (repaso)

Es una terna de algoritmos

Gen: $() \rightarrow K$	Generador de clave
Enc: $K \times P \rightarrow C$	Cifrado
Dec: $K \times C \rightarrow P$	Descifrado

Propiedad básica:

Para todo m y k válidos: $d_k(e_k(m))=m$

Conjuntos involucrados:
K, espacio de claves es el conjunto de todas las <u>claves</u> posibles
P, espacio plano, es el conjunto de todos los <u>mensajes</u> posibles
C, espacio cifrado, es el conjunto de todos los <u>mensajes cifrados</u> posibles

Ataques de texto plano escogido

Chosen Plain Text indistinguishability: $CPA_{A,\Pi}$

Dado un adversario A, y un Criptosistema Π:

1) Se genera una clave $k \leftarrow K$
2) A obtiene $f(x)=e_k(x)$ y genera (m_0, m_1)
3) Se genera $b \leftarrow \{ 0, 1 \}$
4) A recibe $c = e_k(m_b)$
5) A emite $b' = \{ 0, 1\}$

$CPA_{A,\Pi} = 1$ si $b = b'$ (A gana)

Si $Pr[CPA_{A,\Pi}=1] = 0.5 + \varepsilon \Rightarrow \Pi$ es indisting.

Criptosistemas CPA-Secure

Criptosistemas de flujo con tweaks

- Utilizan un parámetro extra, llamado IV para evitar reutilizar exactamente la misma semilla en el generador
- El IV es público pero se selecciona aleatoriamente.

Criptosistemas de bloque

- Con encadenamiento CBC, Counter, OFB, CFB.
- Utilizando primitivas de cifrado de bloque seguras

Un nuevo tipo de ataque

¿Podríamos proteger los sueldos de empleados en una base de datos?

Supongamos usar un criptosistema de flujo CPA-Secure

empleado	sueldo
2542	0xEA26969AA3F61CDD9EC68498DF031CA935EFA9C
2678	0xF4933E107178B88D8EE00F40E43A9A3C9D2EDF79
2789	0x155BBBE7A5442CBBCAB8F57DACF7212255F3641C
2890	0x4D9B47D518F2ED7DE04D601F1856767969A328E8

Un nuevo tipo de ataque

A veces no es necesario conocer el valor para generar problemas…
Supongamos que el atacante conoce su legajo (2678) y el de su jefe (2890)

empleado	sueldo
2542	0xEA26969AA3F61CDD9EC68498DF031CA935
2678	0x4D9B47D518F2ED7DE04D601F1856767969A328E8
2789	0x155BBBE7A5442CBBCAB8F57DACF7212255F3641C
2890	0x4D9B47D518F2ED7DE04D601F1856767969A328E8

¡Aumento de sueldo!

Un nuevo tipo de ataque

La solución no es nada simple.
Considerar cifrar 'empleado||sueldo' para evitar el problema.

empleado	sueldo
2542	0xEA26969AA3F61CDD9EC68498DF031CA935EFA9C
2678	0xF4933E107178B88D8EE00F40E43A9A3C9D2EDF79
2789	0x155BBBE7A5442CBBCAB8F57DACF7212255F3641C
2890	0x4D9B47D518F2ED7DE04D601F1856767969A328E8

Texto plano:
2678 $12,345

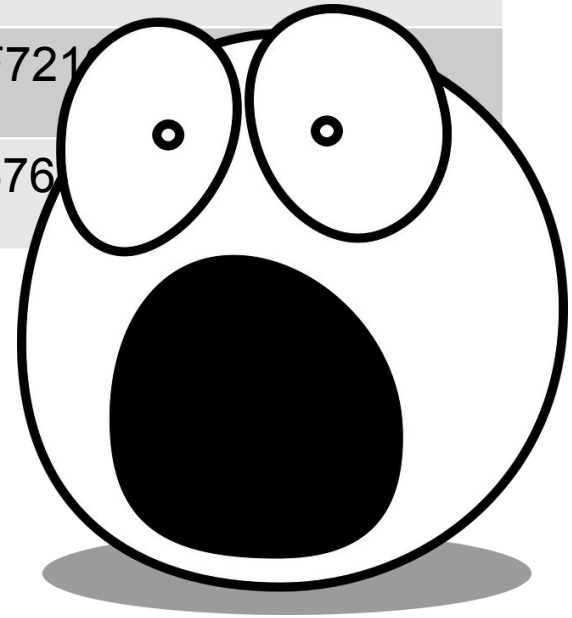
Un nuevo tipo de ataque

¡Modificaciones quirúrgicas aún permiten cambiar el dato!

empleado	sueldo
2542	0xEA26969AA3F61CDD9EC68498DF031CA935EFA
2678	0xF4933E107178B88D8EE00F40E43A9A3C9D2F69F0
2789	0x155BBBE7A5442CBBCAB8F57DACF721
2890	0x4D9B47D518F2ED7DE04D601F185676

Malleability

Texto plano:
2678 **$100,000**



Backstage del ataque

Se necesita conocer el criptosistema y formato de mensaje:

Formato del campo: IV || empl || sueldo

IV = byte[] (12 bytes)

empl = int (4 bytes)

sueldo = int (4 bytes)

```
0x    F4933E107178B88D8EE00F40    E43A9A3C    9D2EDF79
```

Backstage del ataque

Considerar que:

$Enc_k(m) = G(k) \oplus m = c$

Definiendo $c' = c \oplus x$

$Dec_k(c') = m \oplus x$

Avance iterativo hasta el resultado:

2EDF79	00101110	11011111	01111001
xor			
12345	00000000	00110000	00111001
xor			
100000	00000001	10000110	10100000
=			
2F69F0	00101111	01101001	11110000

$Dec_k(c) = 12345$

$Dec_k(c') = 0...00$

$Dec_k(c'') = 100000$

Ataques de texto cifrado escogido

Chosen Ciphertext Attack: $CCA_{A,\Pi}$

Dado un nivel de seguridad n, un adversario A, y un Criptosistema $\Pi(n)$:

1) Se genera una clave $k \leftarrow K$
2) A obtiene $f(x)=e_k(x)$ y $g(x) = d_k(x)$, y genera (m_0, m_1)
3) Se genera $b \leftarrow \{0, 1\}$
4) A recibe $c = e_k(m_b)$, y no puede calcular $g(c)$
5) A emite $b' = \{ 0, 1\}$

$CCA_{A,\Pi} = 1$ si $b = b'$ (A gana)

Si $Pr[CCA_{A,\Pi}=1] < ½ + neg(n) \Rightarrow \Pi$ es indisting.

Ataques de texto cifrado escogido

Ningún criptosistema visto hasta el momento es CCA-Secure

Ejercicio: Demostrar que el cifrado de flujo en general no es CCA-Secure

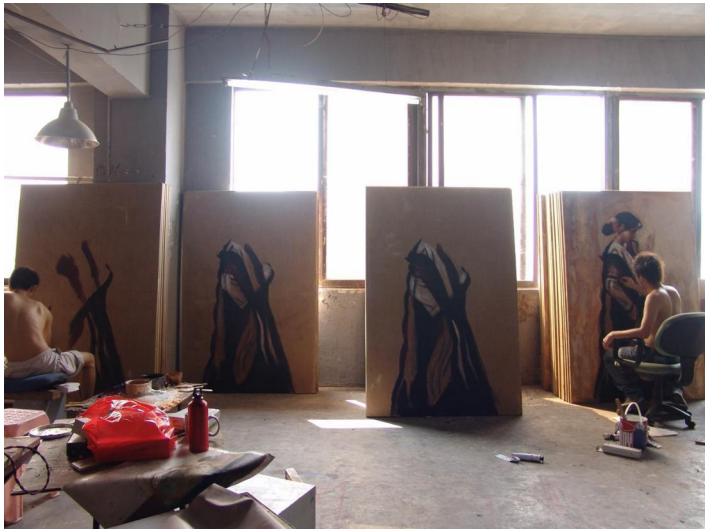
Ayuda: Considerar m0 = (0...0) y m1 = (1...1) y verificar que consultas se pueden hacer a g(x) al recibir c.

Ataques de texto cifrado escogido

Es un problema no resuelto por el cifrado solamente.

Requiere control de integridad

- Consiste en identificar adulteraciones
- Primitiva criptográfica: MAC
  - MAC = Message Authentication Code



Message Authentication Code

Es una terna de algoritmos

Gen: $() \rightarrow K$	Generador de clave
Mac: $K \times P \rightarrow T$	Etiquetador
Vrfy: $K \times P \times T \rightarrow \{0, 1\}$	Verificador

Propiedad básica:

Para todo m y k válidos: $Vrfy_k(m, Mac_k(m)) = 1$

Conjuntos involucrados:
K, espacio de claves es el conjunto de todas las <u>claves</u> posibles
P, espacio plano, es el conjunto de todos los <u>mensajes</u> posibles
T, espacio etiquetas, es el conjunto de todas las <u>etiquetas</u> posibles

Seguridad de un MAC

Message Authentication Experiment: Mac-forge$_{A,\Pi}$

Dado un nivel de seguridad n, un adversario A, y un Criptosistema Π(n):

1) Se genera una clave $k \leftarrow K$
2) A obtiene $f(x)=Mac_k(x)$
3) A realiza las evaluaciones que quiera de f(x)

   (Llámese Q al conjunto de las evaluaciones)
4) A emite (m, t) donde m no pertenece a Q

Mac-Forge$_{A,\Pi}$ = 1 si $Vrfy_k(m, t) = 1$

Si Pr[Mac-Forge$_{A,\Pi}$=1] <= neg(n) => Π es infalsificable

Seguridad de un MAC

Observaciones

El adversario puede obtener un MAC para cualquier mensaje que elija

Se considera roto el MAC si el adversario puede falsificar cualquier mensaje, independientemente de si tiene sentido o no.

Ejercicio

Considerar la seguridad de los siguientes Macs:

$Mac_k(m) = G(k) \oplus m$ (G(-) gen. Pseudoaleat.)

$Mac_k(m) = k \oplus first_k_bits(m)$

$Mac_k(m) = Enc_k(|m|)$ (Enc, CPA-Secure)

Cómo construir un MAC

A partir de una primitiva de cifrado de bloque:

Sea F una función pseudoaleatoria

CBC-MAC (tamaño fijo de mensajes):

Gen: $k \leftarrow \{0, 1\}^n$

Mac:

sea $m = m_1 || m_2 || m_3 ... || m_j$, Sea $t_0 = 00...0$

$t_i = F_k(t_{i-1} \oplus m_i)$.

$Mac_k(m) = t_j$

Vrfy: $Vrfy_k(m, t) = 1 \leftrightarrow t = mac_k(m)$

CBC-MAC

La Construcción anterior es infalsificable SOLO si se permiten mensajes de una misma longitud

Ataque que vulnera el uso de long. variable:

Crear dos bloques aleatorios A y B

$m_1 = A||B$, $m_2 = A$

calcular $t_1 = f(m_1)$, $t_2 = f(m_2)$

Emitir $(A||B||(A \text{ xor } t_1), t_2)$

CBC-MAC

Extensiones seguras para mensajes arbitrarios:

Computar $k' = F_k(|m|)$

Calcular $T_i = F_{k'}(t_{i-1} \oplus m_i)$. $MAC_k(m) = t_j$

Computar $m' = |m| \,||\, m$

$Mac_k(m) = CBC\text{-}MAC_k(m')$

Generar dos claves k_1 y k_2

Calcular $t' = CBC\text{-}MAC_{k1}(m)$

$t = F_{k2}(t')$

CBC-MAC

Utilizar la longitud del mensaje como sufijo NO es seguro:

$m' = m \,||\, |m|$

$Mac_k(m) = CBC\text{-}MAC_k(m')$

Algoritmo atacante:

Crear bloques aleatorios A, B y C

m_1=AAA	$\rightarrow m_1$'=AAA3,	t1=f(m1)
m_2=BBB	$\rightarrow m_2$'=BBB3,	t2=f(m2)
m_3=AAA3CC	$\rightarrow m_3$'=AAA3CC6,	t3=f(m3)

Sea $X = t_1$ xor t_2 xor C

Emitir (BBB3XC, t_3)

Funciones de hash criptograficas

Son pares de algoritmos:

- Gen: $s \leftarrow S$
- Hash: $h = H^s(m) \varepsilon \{0,1\}^L$
  - L es la longitud del hash

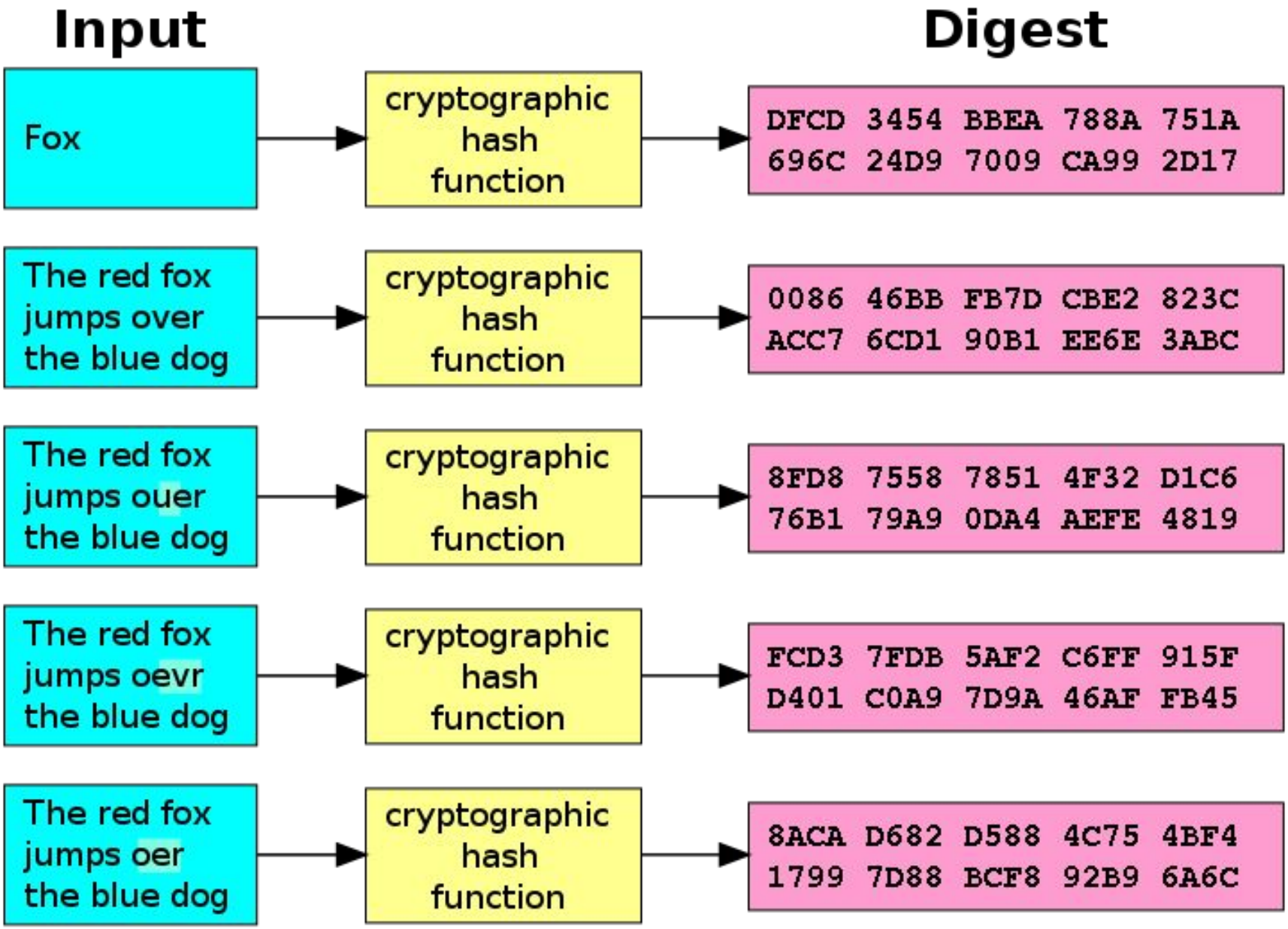
Diferencia importante:

- s no es una clave. Simplemente es un selector.
- En muchas implementaciones $S = \{ s_0 \}$
  - Solo hay una función de hash en la familia

Se las llama funciones de resumen

Análogas a los MACs, pero sin clave

Etiquetadores universales



Colisiones

Colisión:

$x \neq x'$ y $h(x)=h(x')$

Si hay n+1 mensajes y n valores de salida, existe al menos una colisión

Propiedades (definición informal)

Resistencia a preimágenes

Para todo y, es computacionalmente imposible hallar x / $h(x) = y$

Propiedades (definición informal)

Resistencia a segundas imágenes

Para todo x, es computacionalmente imposible hallar $x' \neq x$ / $h(x')=h(x)$

Propiedades (definición informal)

Resistencia a colisiones

Es computacionalmente imposible hallar x, x' / h(x) = h(x') y x ≠ x'

Resistencia a colisiones

Collision resistance: $\text{Hash-Coll}_{A,H}$

Dado un nivel de seguridad n, un adversario A, y una familia de funciones de hash $H^x(n)$:

1) Se selecciona una función $s \leftarrow S$
2) A obtiene acceso a $H(x) = H^s(x)$
3) A emite x, x'

$\text{Hash-Coll}_{A,H} = 1$ si $x \mathrel{!=} x'$ y $H(x)=H(x')$

Si $\Pr[\text{Hash-Coll}_{A,H} = 1] < \text{neg}(n)$, la función H es libre de colisiones

Modelo general iterativo

Propuesto por Merkle en 1989

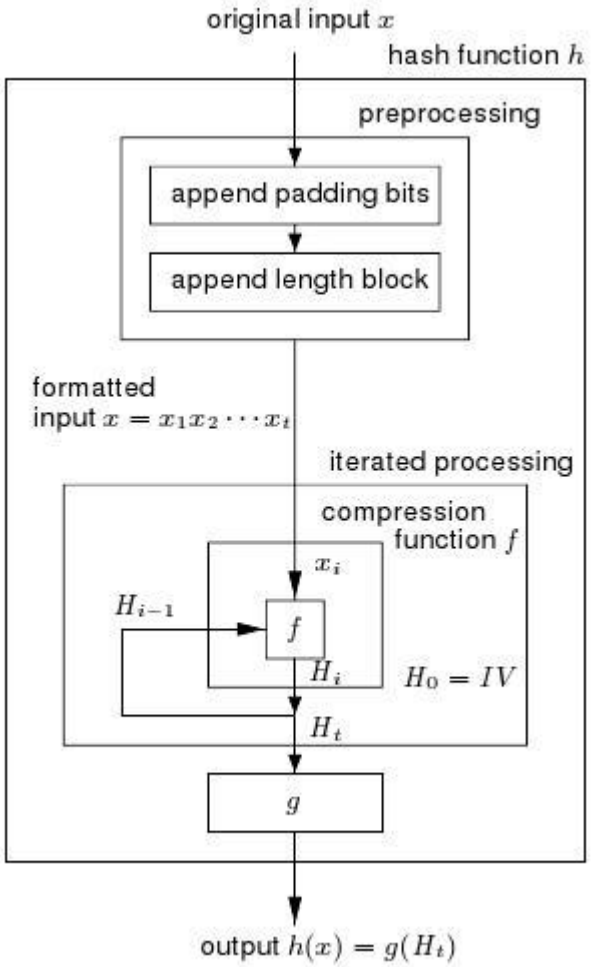
Utilizado por muchas funciones

Preprocesamiento

- Ajustar el tamaño del mensaje
- Agregar bloque con tamaño

Función de compresión

- Similar al hash pero opera sobre bloques pequeños
- La seguridad está dada por la función de compresión



MD5

Entrada: secuencia de hasta 2^{64} bits

Salida: secuencia de 128 bits

Aplicación iterativa

Ejemplos:

“” -> `d41d8cd98f00b204e9800998ecf8427e`

“a” -> `0cc175b9c0f1b6a831c399e269772661`

“abc” -> `900150983cd24fb0d6963f7d28e17f72`

SHA-1

Entrada: secuencia de hasta 2^{64} bits

Salida: secuencia de 160 bits

Aplicación iterativa

Se construye sobre la base de MD5

Ejemplos:

“” -> `da39a3ee5e6b4b0d3255bfef95601890afd80709`

“a” -> `86f7e437faa5a7fce15d1ddcb9eaeaea377667b8`

“abc” -> `a9993e364706816aba3e25717850c26c9cd0d89d`

SHA-3

Entrada: secuencia de hasta 2^{64} bits

Salida: secuencia de 224,256,384 o 512 bits

Estandarizado por NIST en 2013

Aplicación modelo esponja

Ejemplos:

“” -> `a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a`

“a” -> `80084bf2fba02475726feb2cab2d8215eab14bc6bdd8bfb2c8151257032ecd8b`

“abc” → `3a985da74fe225b2045c172d6bd390bd855f086e3e9d525b46bfe24511431532`

Funciones de Hash y MACs

Es posible construir un MAC a partir de una función de hash

HMAC:

Dado H(x) función de hash libre de colisiones

HMACk(x) = {

GEN: $k \leftarrow K$, $s \leftarrow S$

MAC: $t = H^s((k \oplus opad) \,||\, H^s((k \oplus ipad) \,||\, m))$

opad = 0x36...36 ipad= 0x5c5c...5c

}

Es infalsificable (MAC-Forge)

Seguridad de funciones de Hash

Objetivos del atacante. Sea $h: A \rightarrow B$

Preimágenes: dado $y \varepsilon B$, hallar $x / h(x) = y$

Búsqueda por fuerza bruta: requiere $|B|$ intentos

Segundas imágenes: dado $(x,y) / h(x)=y$, hallar $x' / h(x')=y$

Búsqueda por fuerza bruta: requiere $|B|$ intentos

Colisiones: hallar $x, x' / h(x)=h(x')$

Búsqueda por fuerza bruta: requiere $|B|^{1/2}$

Paradoja del cumpleaños

Funciones de hash en la práctica

Tamaño mínimo de salida: 160 bits

- Complejidad de un ataque por fuerza bruta:
  - Imágenes: 2^{160} operaciones
  - Colisiones: 2^{80} operaciones

Primitivas recomendadas

- ~~MD5 → 128 bits~~ Quebrada
- SHA-1 → 160 bits
- SHA-2 → 256/384/512 bits
- SHA-3 → 256/384/512 (Antes conocida como Kekkak)
  - Estándar recomendado para nuevos proyectos

Privacidad e integridad

Tres formas (solo dos seguras):

- ~~Cifrar y autentificar:~~

  ~~$c \leftarrow Enc_{k1}(m),\ t \leftarrow Mac_{k2}(m)$~~

  t puede brindar información de m

- Autentificar, luego cifrar

  $c \leftarrow Enc_{k1}(m \,||\, Mac_{k2}(m))$,

  Puede ser seguro, requiere prueba de seguridad

- Cifrar, luego Autentificar

  $c \leftarrow Enc_{k1}(m),\ t \leftarrow Mac_{k2}(c)$

  Siempre es seguro

Cifrado Autenticado

Combina cifrado y control de integridad

Sean $\Pi_e(Gen_e, Enc, Dec)$ un criptosistema CPA-Secure y $\Pi_m(Gen_m, mac, vrfy)$ un mac infalsificable.

Se define el criptosistema:

Gen: $k_1 \leftarrow Gen_e$, $k_2 \leftarrow Gen_m$

Enc: $c \leftarrow Enc_{k1}(m)$, $t \leftarrow Mac_{k2}(c)$

Dec: Si $vrfy_{k2}(c, t) = 1 \rightarrow m = Dec_{k1}(c)$

Sino $\rightarrow m = /$ (fallo)

El criptosistema resultante es CCA-Secure

Encadenamiento CCM

Significa Counter with CBC-MAC

- "Authenticate-then-encrypt"
- $e_k(\text{cbc-mac}_k(M) \mid\mid m)$

Existe una prueba de seguridad específica

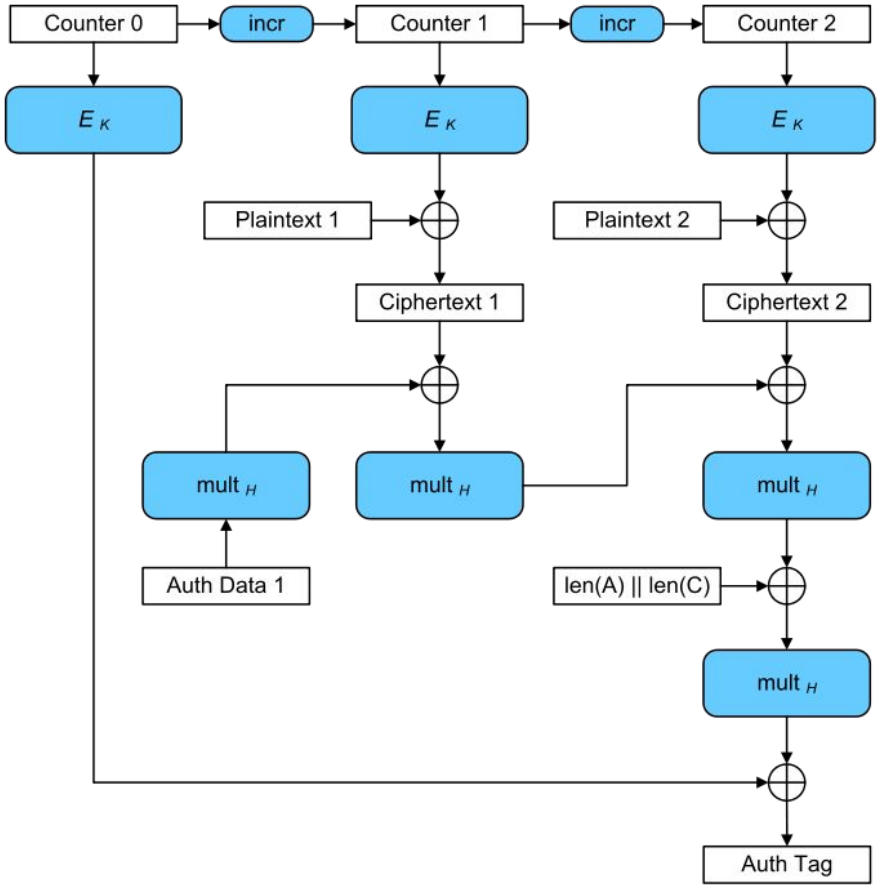
- Si el IV y el nonce no coinciden ni se reutilizan, la construcción es CCA-Secure usando LA MISMA CLAVE
- (ver material adicional en Campus).

Ejercicio, esquematizar un cifrado utilizando AES-CCM

Encadenamiento GCM

Forma de encadenar un criptosistema de bloque

Provee cifrado autenticado



Cifrado modo counter

Auth tag (MAC) GHASH: $GF(2^{128})$

$H = E_k(0000...0000)$

$Mult_h(x) = Mult(x, h)$

$Mult(x, y) = x * y \bmod x^{128}+x^7+x^2+x+1$

Aplicaciones prácticas

En términos generales, sólo utilizar cifrado autenticado en sistemas reales

- El cifrado “normal” puede ser manipulado
- En aplicaciones reales, el requerimiento de privacidad lleva implícito el de integridad
- Muy pocas veces se tiene control sobre el material que va a ser cifrado y descifrado. La solución debería funcionar independientemente de ésto.

Lectura Recomendada

Capítulo 4

Introduction to Modern Cryptography
Katz & Lindell